

フォトリフレクタの動作実験と基板製作（3.3V駆動版）

これまでの実習では、マイコンから信号を出力して「動かす」側（アクチュエータ）の回路を扱ってきました。

ここでは、物体が近づいたことや床面が「白か黒か」を非接触で検出できる光センサー「フォトリフレクタ（RPR-220）」を使用し、回路計算からプログラミング、最終的なハンダ付け実装までの一連のプロセスを学びます。

1. フォトリフレクタの仕組みと、LEDの電流制限抵抗の計算

そもそもフォトリフレクタ（RPR-220）とは？

黒い樹脂のパッケージの中に、以下の「2つの目」が並んで入っているセンサーです。

- 光を出す目（赤外発光ダイオード）：目に見えない赤外線を出します。
- 光を受ける目（フォトトランジスタ）：跳ね返ってきた赤外線を受け取ります。

【白と黒の見分け方】

- 白い面（光を反射する）⇒出した赤外線が跳ね返って戻ってくる。
- 黒い面（光を吸収する）⇒出した赤外線が吸収されて戻ってこない。

① LED側のデータシートを確認する

「光を出す目」を壊さずに光らせるため、適切な **電流制限抵抗の値** を計算します。

- 赤外LEDの順方向電圧（ V_F ）：約 **1.3 V**
- 流したい順方向電流（ I_F ）：推奨値の **10 mA（0.01 A）** とします。

② 3.3V駆動における抵抗値の計算

今回は電源電圧として、**Raspberry Pi Pico** から供給される **3.3 V** を使用します。抵抗にかかる電圧 V_R は、電源電圧からLEDの電圧を差し引いたものです。

$$V_R = 3.3 \text{ V} - 1.3 \text{ V} = 2.0 \text{ V}$$

オームの法則（ $R = \frac{V}{I}$ ）より、必要な抵抗値 R を計算します（電卓を使いましょう）。

$$R = \frac{2.0 \text{ V}}{0.01 \text{ A}} = 200 \Omega$$

💡 **固定抵抗の選定**：計算値は 200Ω です。カラー抵抗の 200Ω （赤黒茶金）または、近い値の 220Ω （赤赤茶金）を使用します。

2. NPN トランジスタの構造とフォトトランジスタの動作

「光を受ける目」であるフォトトランジスタは、**NPN型バイポーラトランジスタ** と全く同じ構造です。違いは、「ベース（B）端子の代わりに、光を受ける窓がある」という点だけです。

- 光が当たっていないとき（黒い床）⇒スイッチは **【OFF】**。
- 光が当たったとき（白い床）⇒光のエネルギーでベースに電流が生まれ、スイッチが **【ON】**。

3. 地面（反射面）との適切な距離を認識しよう

データシートにある「**相対コレクタ電流 — 検出距離**」のグラフを確認してください。

- **RPR-220のベストな検出距離: 約 6 mm**

4. コレクタの負荷抵抗と「感度」の調整

フォトトランジスタがON/OFFしたときの变化を「電圧の変化 (High / Low)」として取り出すために、コレクタ側に **負荷抵抗 (プルアップ抵抗)** を接続します。まずはブレッドボードの実験として、ここに **可変抵抗 (半固定ボリューム)** を使用します。

回路の接続イメージ (実験用)

```
+3.3V (Picoの3V3ピン)
|
[ 可変抵抗 (10k~50kΩ等) ] ← ★ここを回して感度を調整する
|
+-----> 出力電圧 (ここをAD3で観測する)
|
[ C ] コレクタ (RPR-220 受光側: NPN構造)
[ E ] エミッタ
|
GND (0V)
```

5. AD3 (Analog Discovery 3) による「感度調整」の実験

⚠ **重要: AD3の電源電圧を 3.3V に設定すること** AD3の初期設定 (5V) のまま回路を動かすと、後でPicoに繋いだときにマイコンが壊れます。

実験手順

1. ブレッドボードに回路を配線します。
2. WaveForms の「**Supplies**」画面を開き、**V+ の電圧を「3.3 V」に変更**してからONにします。
3. AD3のオシロスコープ機能を開き、**1+** を出力電圧ピン、**1-** をGNDへ接続します。
4. センサーを床面 (約6mmの高さ) に近づけ、「白い紙」と「黒いテープ」を交互にかざします。
5. ★**感度調整**★: 可変抵抗のツマミを回し、「**白い紙で 0V付近 (Low)**」、「**黒いテープで 3.3V付近 (High)**」まで波形がしっかりと上下するポイントを見つけてください。

6. マイコン (Pico) の ADC による判定プログラミング

実際のロボットに搭載する本番の基板では、安全確保と省スペース化のため、**可変抵抗を使わず「10kΩの固定抵抗」**を使用します。現場での微調整 (キャリブレーション) は、マイコンの **ADC (アナログ入力) 機能** を使ってプログラム側で行います。

手順①: 回路の変更と数値の確認

1. ブレッドボードの「可変抵抗」を外し、「**10kΩの固定抵抗**」に差し替えます。
2. 電源を AD3 から **Picoの 3V3ピン と GNDピン** に繋ぎ替えます。

3. センサーの出力ピンを、Picoの **GP26 (ADC0)** に接続します。
4. 以下のプログラムを実行し、コンソール (Shell) の数値を確認します。

```
import machine
import time

sensor = machine.ADC(machine.Pin(26))

while True:
    val = sensor.read_u16() # 0~65535の数値を読み取る
    print("ADC Value:", val)
    time.sleep_ms(100)
```

手順②：しきい値判定プログラム（ミニミッション！）

読み取った数値をもとに、「黒い線を踏んだら、Pico本体の緑色LEDを点灯させる」プログラムを完成させなさい。

```
import machine
import time

sensor = machine.ADC(machine.Pin(26))
led = machine.Pin("LED", machine.Pin.OUT) # Pico内蔵LED

# 白と黒を区別するための「しきい値」を自分で決める
THRESHOLD = (        ) # 例：30000 など、白と黒の中間の数字を入れる

print("Sensor Ready!")

while True:
    val = sensor.read_u16()

    # センサーの値がしきい値より大きい（＝黒）なら
    if val > THRESHOLD:
        print("LED ON")
        led.value(1) # LED 点灯
    else:
        print("LED OFF")
        led.value(0) # LED 消灯

    time.sleep_ms(50)
```

- ☐ プログラムが正しく動き、白黒でLEDがチカチカ切り替わったら先生に見せてチェックを受ける。